

1. What is Terraform?

Terraform is an Infrastructure as Code (IaC) tool developed by HashiCorp used to provision and manage cloud infrastructure using configuration files.

2. What is Infrastructure as Code (IaC)?

Infrastructure as Code means managing and provisioning infrastructure using code instead of manual configuration.

3. What are Terraform providers?

Providers are plugins that allow Terraform to interact with cloud platforms such as Azure, AWS, and Google Cloud.

4. What is a Terraform resource?

A resource represents infrastructure components like virtual machines, storage accounts, networks, etc.

5. What is the Terraform workflow?

The typical workflow is terraform init, terraform plan, terraform apply, and terraform destroy.

6. What does terraform init do?

It initializes the working directory, downloads providers, and configures the backend.

7. What does terraform plan do?

It shows an execution plan of what Terraform will create, modify, or destroy.

8. What does terraform apply do?

It executes the changes defined in the Terraform configuration.

9. What does terraform destroy do?

It removes all infrastructure resources managed by Terraform.

10. What is terraform validate?

It checks if Terraform configuration files are syntactically valid.

11. What is terraform fmt?

It formats Terraform configuration files into a standard readable format.

12. What is a Terraform state file?

A state file keeps track of infrastructure resources managed by Terraform.

13. Why is the state file important?

It maps Terraform configuration to real infrastructure and helps Terraform track resources.

14. What is a Terraform backend?

Backend defines where the Terraform state file is stored.

15. What is remote backend?

A remote backend stores the state file in remote storage such as Azure Storage or AWS S3 for team collaboration.

16. What are variables in Terraform?

Variables allow you to make Terraform configurations reusable and dynamic.

17. What is terraform.tfvars?

It contains values assigned to variables declared in variables.tf.

18. What are Terraform outputs?

Outputs display important information after infrastructure deployment.

19. What is a Terraform module?

A module is a reusable set of Terraform configuration files.

20. Why use Terraform modules?

Modules improve reusability, maintainability, and organization of Terraform code.

21. What are .tf files?

Files with .tf extension contain Terraform configuration code.

22. What is terraform show?

It displays the current Terraform state or execution plan in a human-readable format.

23. What is terraform graph?

It generates a visual dependency graph of Terraform resources.

24. What is terraform version?

It shows the installed Terraform version.

25. What is count in Terraform?

count is used to create multiple instances of a resource.

26. What is for_each?

for_each is used to create multiple resources using maps or sets.

27. Difference between count and for_each?

count uses index numbers while for_each uses key-value pairs.

27. What is a data source?

Data sources allow Terraform to fetch information about existing resources.

29. Difference between resource and data source?

Resource creates infrastructure, while data source reads existing infrastructure.

30. What are Terraform functions?

Functions are built-in operations used to manipulate values.

31. What is lookup() function?

lookup() retrieves a value from a map.

32. What is length() function?

length() returns the number of elements in a list or characters in a string.

33. What is join() function?

join() combines list elements into a single string.

34. What is split() function?

split() divides a string into a list.

35. What is Terraform workspace?

Workspaces allow multiple environments using the same Terraform configuration.

36. What are workspace commands?

terraform workspace list, terraform workspace new, terraform workspace select.

37. What is terraform refresh?

It updates the Terraform state file with the real infrastructure state.

38. What is terraform output?

It displays values defined in output variables.

39. What is Terraform project structure?

A typical structure includes main.tf, variables.tf, outputs.tf, and modules folder.

40. What is terraform import?

terraform import brings existing infrastructure into Terraform state.

41. Example use of terraform import?

Used when resources already exist but need to be managed by Terraform.

42. What is Terraform state locking?

It prevents multiple users from modifying the state simultaneously.

43. How is state locking implemented in Azure?

Using Azure Storage blob locking mechanism.

44. What is terraform state list?

Lists all resources in the Terraform state.

45. What is terraform state show?

Shows detailed information about a resource in the state.

46. Difference between Terraform and Ansible?

Terraform is mainly for provisioning infrastructure while Ansible is for configuration management.

47. What is Terraform lock file?

.terraform.lock.hcl stores provider dependency versions.

48. What is Terraform CI/CD?

Using pipelines to automate Terraform deployment.

49. How to integrate Terraform with Azure DevOps?

Using Terraform tasks in YAML pipelines.

50. What are Terraform best practices?

Use modules, remote state, version locking, and proper folder structure.

51. How do you manage multiple environments?

Using workspaces or separate tfvars files.

52. What happens if Terraform state is deleted?

Terraform may recreate infrastructure since it loses tracking.

53. What is Terraform execution plan?

A preview of changes Terraform will make.

54. How do you debug Terraform errors?

Using terraform validate, terraform plan

55. How do you organize Terraform code?

Using modules, environment folders, and clear naming conventions.

56. What is Terraform pipeline?

A CI/CD pipeline that automates Terraform deployment.

57. How do you scale infrastructure?

Using count or for_each.

58. What is resource targeting?

Using -target option to apply changes to specific resources.

59. What is destroy targeting?

Using -target option to destroy specific resources.

60. Why is Terraform popular?

Because it enables automated, version-controlled, and multi-cloud infrastructure provisioning.

61. What is Terraform and why is it used in Azure?

Terraform is an Infrastructure as Code (IaC) tool used to automate provisioning and management of Azure resources in a consistent and repeatable manner.

62. What are Providers? How does AzureRM provider work?

The AzureRM provider is the plugin that allows Terraform to interact with Azure services.

63. What is Terraform State?

Terraform maintains a state file that keeps track of resources created in Azure.

64. Where should Terraform state be stored in Azure?

Azure Storage Account → Blob Container (terraform.tfstate file).

65. What is a Terraform backend?

Backend defines where Terraform state is stored (local, Azure blob, S3, etc.).

2. Terraform State & Locking

66. What is Terraform State Lock?

Prevents multiple users from running Terraform at the same time, avoiding conflicts.

67. How does Azure handle state lock?

Azure uses a **Blob lease** to lock Terraform state.

68. How to unlock a stuck state lock?

Use CLI to break blob lease:

az storage blob lease break --container-name tfstate --blob terraform.tfstate

3. Terraform Files & Structure

69. What are .tf and .tfvars files?

- .tf: Terraform configuration files
- .tfvars: Variable definitions

70. Structure of common Terraform files

- **main.tf** → Resources
- **variables.tf** → Variable definitions
- **outputs.tf** → Outputs

71. What is terraform.tfstate and terraform.tfstate.backup?

State file and its auto-generated backup.

72. Most Common Terraform Commands

Common Commands:

terraform init

terraform plan

terraform apply

terraform destroy

terraform fmt

terraform validate

terraform refresh terraform import

73. terraform init

Initializes backend and downloads providers.

73. terraform plan

Shows the execution plan.

74. terraform apply

Applies the changes.

5. Terraform in Azure DevOps

75. How to run Terraform in Azure DevOps pipelines?

Using tasks for:

- Installing Terraform
- Azure CLI login
- Running init/plan/apply

76. How to secure backend credentials?

Store secrets in **Azure Key Vault**.

6. Terraform Azure Resources

77. How to reference Azure resources in Terraform?

`azurerm_resource_group.rg.name`

78. How to define dependencies?

- Implicit: via resource references
 - Explicit: `depends_on = []`
-

7. Terraform Modules

79. What is a Terraform module?

Reusable infrastructure components.

80. How to call a module?

```
module "network" {  
  source = "./modules/vnet"  
}
```

81. What are Registry modules?

Modules available publicly on Terraform Registry.

8. Provisioners

82. What are provisioners?

Scripts used to configure VMs (not recommended).

83. remote-exec vs local-exec

- remote-exec: runs inside the VM
 - local-exec: runs on local machine
-

9. Terraform Import

84. How do you import existing Azure resources?

```
terraform import azurerm_resource_group.rg \  
/subscriptions/<id>/resourceGroups/myRG
```

85. Does Terraform generate .tf files after import?

No. You must manually write the .tf configuration.

10. Common Terraform Errors

86. Resource already exists

Fix → Use terraform import.

87. Provider version conflict

Fix → Declare provider version in required_providers.

88. Invalid index / null value

Occurs due to incorrect variable values.

11. Azure-Specific Questions

89. How to authenticate Terraform to Azure?

- Azure CLI
- Service Principal
- Managed Identity

90. AzureRM provider configuration

```
provider "azurerm" {  
  
  features {}  
  
}
```

91. What is the features {} block?

A mandatory block for AzureRM provider, even if empty.

Terraform Theory Overview

1. Terraform Init

Terraform init is the first command you run in any new or existing Terraform configuration directory. It initializes the working directory and sets up the backend, provider plugins, and modules.

- Downloads required provider plugins.
- Initializes backend configuration (where the state file will be stored).
- Prepares modules.

2. Terraform Validate

Terraform validate checks whether your configuration files are syntactically valid and internally consistent.

- It does **not** connect to cloud providers.
- Useful for detecting errors before running plan.

3. Terraform Plan

Terraform plan shows the changes Terraform will make to reach the desired infrastructure state defined in your configuration.

- Creates an execution plan.
- Shows additions, modifications, and deletions.
- Helps confirm expected behavior before applying.

4. Terraform Apply

Terraform apply executes the changes defined in the plan.

- Creates, updates, or deletes infrastructure.
- Can run with or without a prior plan file.
- Requires confirmation (unless using -auto-approve).

5. Terraform Destroy

Terraform destroy removes all infrastructure managed by Terraform.

- Used to clean up resources.
- Accepts confirmation before destroying.

terraform destroy

terraform destroy -auto-approve

terraform destroy -target="azurerm_storage_blob.example"

6. Terraform State File

Terraform uses a state file (terraform.tfstate) to track the real-world resources it manages.

- Maps Terraform configuration to actual infrastructure.
- Stored locally or remotely (S3, Azure Storage, GCS, etc.).
- Remote state enables collaboration.
- Must be secured because it contains sensitive information.

Purpose of State File

- Keeps track of resource metadata.
- Helps Terraform understand **what already exists**.
- Makes plan operations faster.
- Prevents accidental recreation of resources.

7. What is Terraform State Lock?

In Terraform, the **state lock** (often seen as a “lock file”) is a mechanism that **prevents multiple users or processes from modifying the same Terraform state at the same time**.

When you run commands like:

- terraform apply
- terraform plan (sometimes)
- terraform destroy

Terraform creates a **state lock** to ensure that **only one operation** is making changes to the infrastructure.

This lock prevents:

- Two people running apply at the same time
- A pipeline and a developer running Terraform simultaneously
- Corruption of the **terraform.tfstate** file

8. What is a Terraform State Backup File?

A Terraform state backup file is a copy of your previous state, created automatically whenever Terraform updates the main state file.

Its purpose is to provide a safe restore point in case:

- The main terraform.tfstate gets corrupted
- A deployment fails
- You accidentally overwrite the state

9. variables.tf

The variables.tf file is used to declare input variables that make your Terraform configuration flexible and reusable.

- Defines variable names, types, descriptions, and default values.
- Does **not** store actual values.
- Helps parameterize your infrastructure.

Example

```
variable "location" {  
  
  type    = string  
  
  description = "Azure region for resources"  
  
  default  = "eastus"
```

}

10. terraform.tfvars

The terraform.tfvars file contains the **actual values** for variables defined in variables.tf.

- Automatically loaded by Terraform.
- Helps keep real values separate from variable declarations.
- Useful for environment-specific values (dev, test, prod).

Example

```
location = "centralindia"
```

11. Purpose of Using Variables

- Reuse Terraform configuration across multiple environments.
- Avoid hardcoding values.
- Improve maintainability and consistency.

12. Terraform Modules

Terraform modules are reusable groups of resources that help organize and standardize your infrastructure.

What is a Module?

A module is a folder containing Terraform .tf files that define resources, variables, outputs, and logic.

Why Use Modules?

- Reusability (DRY principle)
- Cleaner folder structure
- Easier maintenance
- Standardization across environments

13. Terraform Format (terraform fmt)

terraform fmt automatically formats Terraform configuration files to a standard style.

Purpose

- Improves readability
- Maintains consistent formatting
- Helps teams follow Terraform coding standards

Commands

terraform fmt

14. Terraform Show

terraform show displays the current state or a saved execution plan in a readable format.

Commands

terraform show

terraform show terraform.tfstate

15. Terraform Refresh

terraform refresh updates the Terraform state file with the **real infrastructure state**.

terraform refresh

Purpose

- Synchronizes Terraform state with actual resources.

16. Terraform Graph

terraform graph generates a dependency graph of Terraform resources.

terraform graph

It shows how resources depend on each other.

Used for:

- Understanding infrastructure dependencies
- Visualizing resource relationships

17. Terraform Version

Displays installed Terraform version.

terraform version

18. Terraform Output

terraform output is a command used to display the values of output variables defined in your Terraform configuration. These outputs usually expose important information about the infrastructure Terraform created.

For example, after creating resources like a VM, Storage Account, or Load Balancer, you may want to display values such as:

- Public IP address
- Resource ID
- DNS name
- Storage account name
- Database connection string

Instead of manually checking the cloud portal, Terraform can print these values directly in the terminal.

Terraform Modules

A Terraform Module is a reusable set of Terraform configuration files.

Instead of writing the same infrastructure code repeatedly, we create a module and reuse it for different environments like:

- Dev
- Test
- UAT
- Prod

Explanation:

File	Purpose
main.tf	Infrastructure code

File	Purpose
variables.tf	Variable definitions
modules folder	Reusable infrastructure module
dev.tf	Calling the module for Dev environment